

Apex Grand Prix: 2D Top-View Supercar Racing Simulation

Comprehensive Technical Design, Algorithmic Implementation, and Architecture Report (Project 3)

Course: Computer Graphics (B.Tech Lab)

Graphics Architecture: 100% CPU Software Rasterizer (ARGB32 Pixel Buffer)

Date: October 2026

Framework: Modern C++17, Native Qt 6 GUI

Target FPS: Locked 60 FPS Deterministic Loop

1. Executive Summary & Design Motivation: "Cars Seen Properly"

In traditional pseudo-3D or third-person racing games (such as earlier projects), vehicles are viewed almost exclusively from behind. This severe perspective foreshortening introduces significant geometric drawbacks: the side aerodynamic contours, front intake splitters, steerable wheels, and wide-circuit layouts are compressed into flat rear bumpers. Furthermore, tight camera cones make it difficult for drivers to gauge lateral clearance or predict upcoming curves.

Project 3 (Apex Grand Prix) reimagines the simulation from a true **top-down (bird's-eye) 2D geometric perspective**, allowing every vehicle, wheel rotation, track curvature, and dynamic visual effect to be observed in full mathematical and visual fidelity:

- **Full Aerodynamic Chassis Silhouette:** Distinct hood styling, cockpit roof numbers (#1 to #5), tinted windshields with specular glints, side aero mirrors, and rear GT wings with endplates.
- **Visible Steerable Front Wheels:** Front wheels dynamically rotate by steering angle α relative to the car body using hierarchical 2D affine rotation matrices.
- **Twin Xenon Headlight Cones:** Dynamic alpha-blended light projection cones with distance attenuation cast over asphalt and road markings.
- **Persistent Tire Skid Rubber:** Dual rubber tracks deposited on high-g drifts and heavy deceleration, gradually fading via alpha decay.
- **Multi-Layered Nitro Exhaust Flames:** Ejected rocket thruster flames with animated combustion cores and ember particle showers.
- **Educational Software Rasterizer:** Built strictly with fundamental Computer Graphics algorithms without OpenGL, Vulkan, or DirectX hardware shaders, writing raw pixels directly into a 32-bit linear framebuffer array (`uint32_t *fb`).

Core Laboratory Objective & Technical Scope

Demonstrate complete end-to-end mastery of core Computer Graphics algorithms taught in lecture: Bresenham's Integer Line Drawing, Midpoint Circle/Ellipse, Scanline Triangle Rasterization, 2D Homogeneous Affine Matrices (3x3), Alpha Blending, Catmull-Rom Spline Interpolation, and Autonomous Kinematic Agents, running deterministically at a locked 60 FPS on the CPU.

2. High-Level System Architecture & Subsystem Topology

The project is structured into six decoupled, single-responsibility C++ modules. Clean separation of concerns allows the mathematical rasterizer to remain completely independent of the physics engine and Qt UI scaffolding:

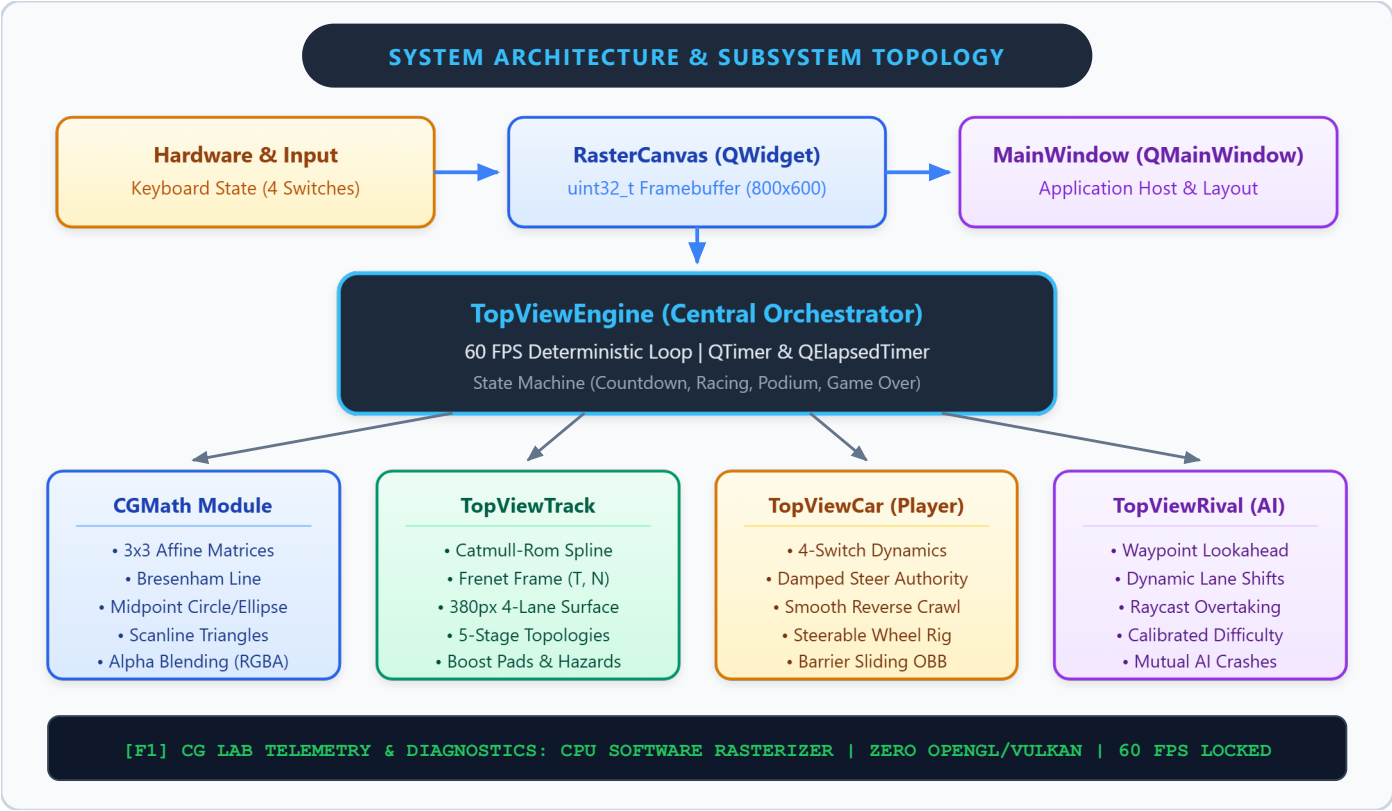


FIGURE 1: HIGH-LEVEL SYSTEM ARCHITECTURE, DATA FLOW, AND MODULE INTERCONNECTIONS

Module Name	C++ Source Files	Primary Responsibility & Architectural Role
RasterCanvas	raster_canvas.h/cpp	Low-level Qt QWidget host. Allocates a linear <code>std::vector<uint32_t></code> ARGB32 framebuffer (800x600). Manages asynchronous keyboard events without auto-repeat latency.
TopViewEngine	engine_topview.h/cpp	Central orchestrator. Maintains a 60 FPS deterministic game loop via QTimer/QElapsedTimer, finite state machine, camera viewport matrices, particle buffers, HUD, and minimap.
CGMath	cg_math.h/cpp	Pure software rasterizer and math library. Provides 3x3 affine transformation matrices (Mat2D), 2D vector algebra (Vec2D), Bresenham line with thickness, Midpoint circle/ellipse, scanline triangles, and alpha blending.
TopViewTrack	track_topview.h/cpp	Procedural track spline generator. Constructs smooth Catmull-Rom curves, computes moving Frenet-Serret normal frames, generates 380px 4-lane meshes, rumble kerbs, boost pads, and checks barrier collisions.
TopViewCar	car_topview.h/cpp	Vehicle planar kinematics. Handles progressive throttle, calibrated steering sensitivity with speed-dependent damping, smooth braking, slow reverse crawling (~35 km/h), steerable wheel rigs, and barrier sliding.
TopViewRival	rival_topview.h/cpp	Autonomous AI racing agent. Executes waypoint lookahead navigation, dynamic lane offset tracking, obstacle raycasting, proactive overtaking, level-based speed progression, and mutual crash dynamics.

3. Component 1: Mathematical Foundations & Software Rasterizer (cg_math)

The rasterizer operates directly on a memory buffer of 800×600 32-bit unsigned integers formatted as 0xAARRGGBB. Every visual element—lines, curves, car silhouettes, wheels, and HUD gauges—is rasterized pixel-by-pixel.

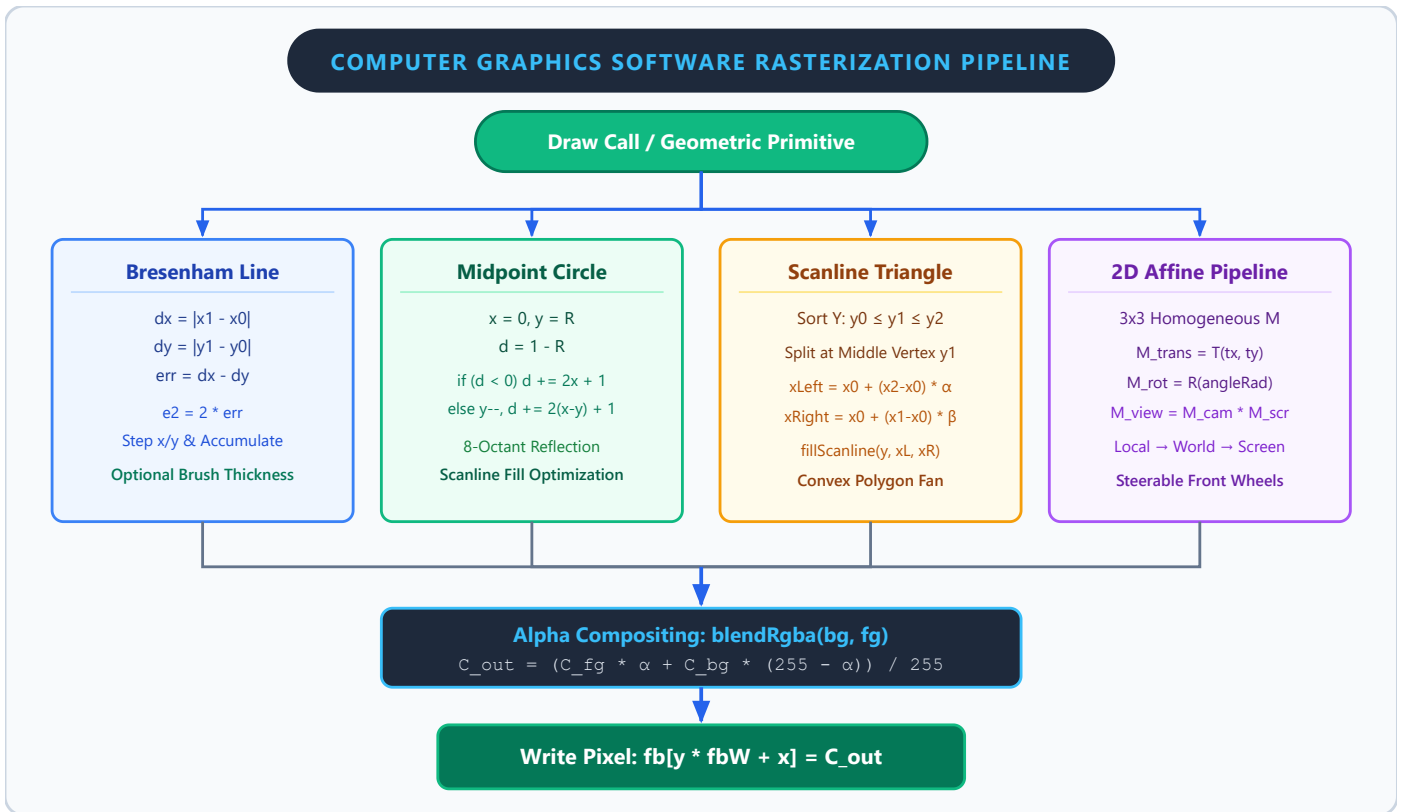


FIGURE 2: ALGORITHMIC SOFTWARE RASTERIZATION PIPELINE (CG_MATH)

3.1. 2D Homogeneous Affine Transformations (3×3 Matrices)

To support seamless composite transformations (translation, rotation, scaling) with a single matrix multiplication, 2D coordinates are represented in homogeneous form $[x, y, 1]^T$. The Mat2D structure implements:

```

Translation Matrix:
T(tx, ty) = [ 1  0  tx ]
             [ 0  1  ty ]
             [ 0  0  1 ]

Rotation Matrix:
R(θ) = [ cos θ  -sin θ  0 ]
        [ sin θ   cos θ  0 ]
        [  0      0     1 ]

Composite Pipeline: M_final = M_Screen × M_Camera_View × M_World × M_Local_Wheel
Point Transform:   p_screen = M_final × p_local
  
```

3.2. Bresenham's Line Algorithm with Variable Thickness

Bresenham's algorithm determines the pixels to be rasterized on a discrete grid using purely integer addition and subtraction, avoiding floating-point math in the hot inner loop. The error accumulator $e = \Delta x - \Delta y$ decides whether to increment the secondary axis:

```

dx = |x1 - x0|, dy = |y1 - y0|, sx = (x0 < x1 ? 1 : -1), sy = (y0 < y1 ? 1 : -1)
err = dx - dy
while (x0 != x1 || y0 != y1):
    putPixelSafe(x0, y0, col) // brush-box convolution applied when thickness > 1
    e2 = 2 * err
    if (e2 > -dy): err -= dy; x0 += sx
    if (e2 < dx): err += dx; y0 += sy
  
```

3.3. Midpoint Circle & Ellipse Algorithms

The Midpoint Circle algorithm evaluates the implicit circle function $F(x, y) = x^2 + y^2 - R^2$ at midpoint steps. Starting at $(0, R)$ with initial decision parameter $d_0 = 1 - R$:

- If $d_k < 0$: Midpoint is inside the circle $\rightarrow d_{k+1} = d_k + 2x_k + 1$ (step East).
- If $d_k \geq 0$: Midpoint is outside or on boundary $\rightarrow d_{k+1} = d_k + 2(x_k - y_k) + 1$, decrement y (step South-East).

- By evaluating only the first octant ($0 \leq x \leq y$), all eight symmetrical octants are plotted simultaneously. In `fillMidpointCircle`, horizontal scanlines connect symmetrical pairs for fast solid rasterization.

3.4. Scanline Triangle Rasterization with Alpha Blending

Convex polygons and vehicle chassis plates are rendered via fan triangulation. Each triangle is sorted vertically by Y-coordinate ($y_0 \leq y_1 \leq y_2$) and split into flat-bottom and flat-top segments. Along each scanline y , left and right spans are interpolated, and pixels are composited using Porter-Duff 'Source Over' alpha blending:

```
invA = 255 - α_fg
R_out = (R_fg · α_fg + R_bg · invA) / 255
G_out = (G_fg · α_fg + G_bg · invA) / 255
B_out = (B_fg · α_fg + B_bg · invA) / 255
```

4. Component 2: Procedural Track Spline Engineering (track_topview)

Race tracks are procedurally generated from sparse designer control points using Catmull-Rom splines, creating seamless, organic road curvature with continuous first derivatives (C^1 continuity).

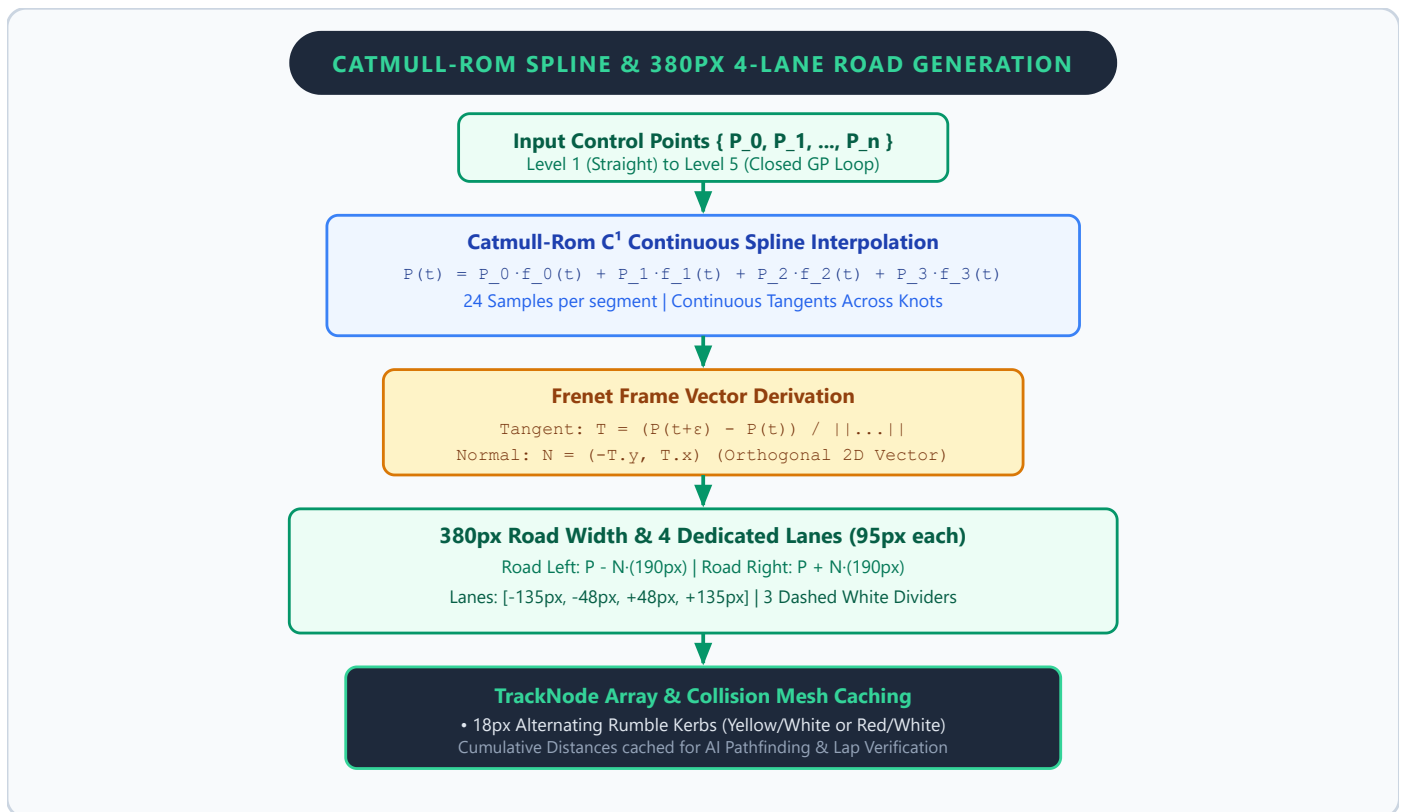


FIGURE 3: PROCEDURAL CATMULL-ROM SPLINE & 380PX 4-LANE TRACK GENERATION

4.1. Catmull-Rom Spline Formulation

Given four consecutive control points P_0, P_1, P_2, P_3 , the interpolated point for $t \in [0, 1]$ is computed via cubic blending functions:

$$P(t) = 0.5 \cdot [(2 \cdot P_1) + (-P_0 + P_2) \cdot t + (2 \cdot P_0 - 5 \cdot P_1 + 4 \cdot P_2 - P_3) \cdot t^2 + (-P_0 + 3 \cdot P_1 - 3 \cdot P_2 + P_3) \cdot t^3]$$

Unlike Bézier curves, Catmull-Rom splines pass directly through all interior control points, making track layout intuitive. Closed circuits wrap indices modulo N , while open drag sprints clamp boundary indices.

4.2. Moving Frenet-Serret Frame & 380px 4-Lane Architecture

At each spline sample, forward tangent T and lateral normal N vectors establish a local coordinate frame:

```
T = normalize(P(t + ε) - P(t)),      N = (-T.y,  T.x)
Road Left Edge:  V_left  = P(t) - N * (W_road / 2)
Road Right Edge: V_right = P(t) + N * (W_road / 2)
```

Massive 380px Road Width: To eliminate crowding and allow free overtaking, the asphalt ribbon spans 380 pixels (over 3.5× wider than conventional tracks), subdivided into four 95px lanes:

- **Lane 1 (Inside Rail):** Lateral offset -135 px (Left boundary to -95 px).
- **Lane 2 (Mid-Left):** Lateral offset -48 px (-95 px to Centerline).
- **Lane 3 (Mid-Right):** Lateral offset +48 px (Centerline to +95 px).
- **Lane 4 (Outside Rail):** Lateral offset +135 px (+95 px to Right boundary).
- **Markings & Kerbs:** Continuous center line, 3 dashed white lane dividers, and 18px alternating bi-color rumble kerbs.

4.3. 5-Level Track Progression Topology

Stage	Track Name	Topology & Geometric Structure	Laps	Environmental Theme
Level 1	Desert Highway Sprint	100% Straight 4-lane drag strip heading directly North (-Y). Zero curvature.	1 Sprint	Golden sand dunes, smooth blacktop, yellow kerbs.
Level 2	Coastal Highway	Gentle flowing sweeping S-curves along the coastline. Non-circular.	1 Sprint	Ocean azure, gold guardrails, coastal palm scatter.
Level 3	Alpine Canyon Pass	Winding technical elevation bends and gorge bridges. Non-circular.	1 Sprint	Mountain slate, deep pine foliage, red safety barriers.
Level 4	Meadow GP Circuit	Full 13-control-point closed-loop circuit with chicanes and hairpins.	3 Laps	Lawn mowing stripes, classic red/white GP kerbs.
Level 5	Neo Tokyo Expressway	High-speed midnight cyber loop with tight apexes and underpasses.	3 Laps	Wet dark asphalt, glowing neon cyan/pink kerbs.

5. Component 3: Vehicle Kinematics, 4-Switch Handling & Tuning (car_topview)

Vehicle physics are modeled using rigid-body planar kinematics balancing throttle propulsive forces, tire friction, aerodynamic drag, and lateral centrifugal slip.

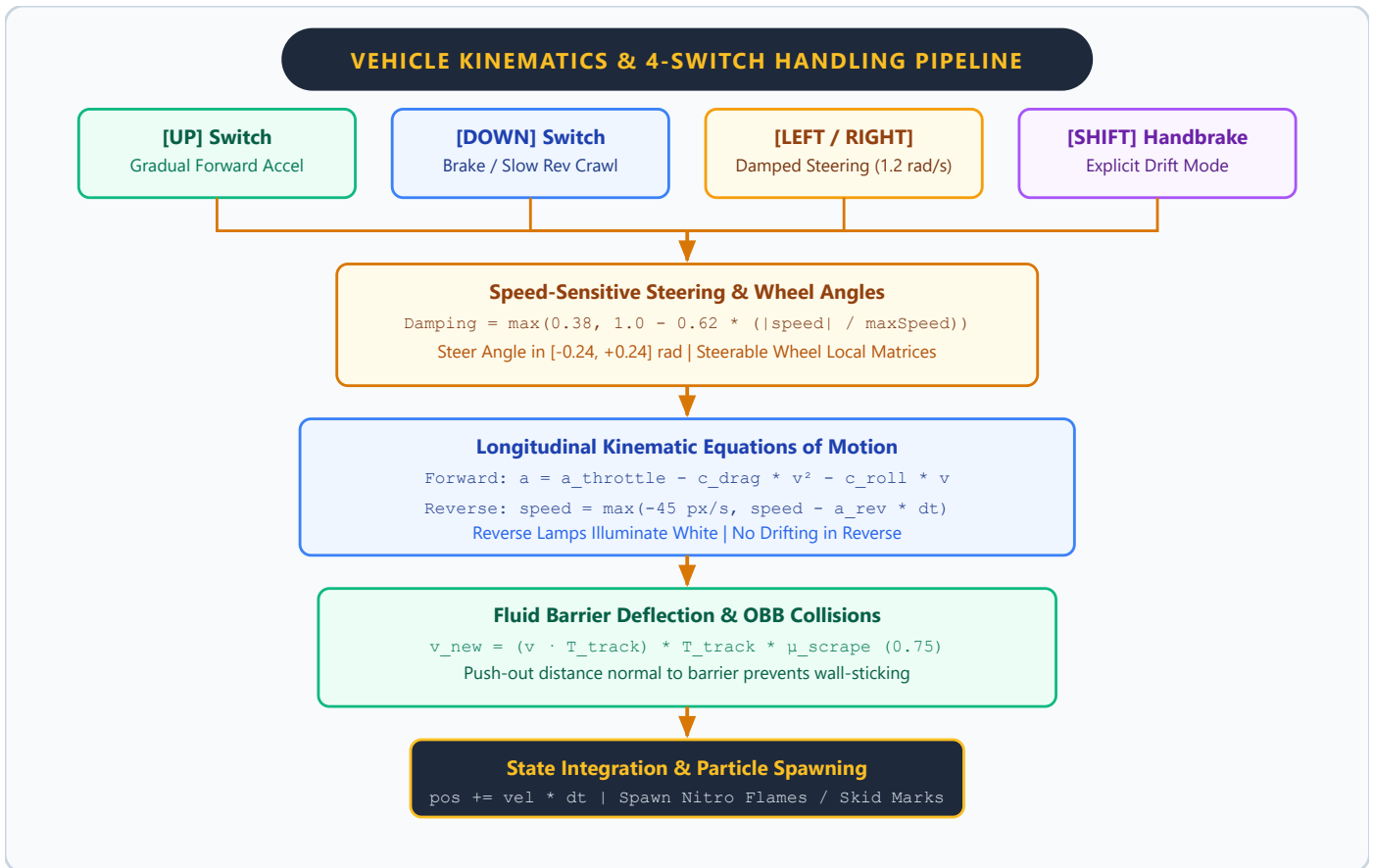


FIGURE 4: VEHICLE KINEMATICS, 4-SWITCH CONTROL SCHEME & BARRIER DEFLECTION

5.1. The 4-Switch Control Scheme Calibration

Based on user playtesting feedback, the 4 primary directional switches were specifically re-engineered:

- **Drive Forward ([UP] / [W]):** Progressive forward acceleration. Rather than instantaneous wheelspin, motor torque ramps smoothly from stationary to top speed (380–430 px/s), giving the driver refined throttle control.
- **Brake / Reverse ([DOWN] / [S]):** Dual-action switch. When traveling forward, it acts as a smooth brake (190 px/s²). When the vehicle comes to a stop or crawls below 5 px/s, it seamlessly engages **controlled slow reverse**, capped at -45 px/s (~35 km/h). Tail lamps blaze into glowing white reverse lights, allowing easy recovery from barriers without disorienting the player.
- **Steer Left / Right ([LEFT] / [RIGHT] / [A] / [D]):** Calibrated low-sensitivity steering. Turn rates were dampened from an erratic 3.5–4.1 rad/s down to 1.10–1.40 rad/s. Furthermore, **speed-sensitive high-speed damping** was implemented:

```
f_damping = max(0.38, 1.0 - 0.62 * (|v| / v_max))
Δθ_frame = turnRate * f_damping * Δt
```

At high velocities, steering authority is automatically attenuated to 38%, preventing sudden oversteer or violent spin-outs on curves!

- **Handbrake Drift ([SHIFT]):** Decoupled drift activation. Previously, pressing Down while steering accidentally triggered drift, cutting lateral grip and doubling sensitivity. In Project 3, drift is strictly decoupled and requires [SHIFT], ensuring ordinary turning always maintains 100% tire grip.

5.2. Steerable Front Wheels via Hierarchical Transforms

When steering, front left and right wheels rotate visually by $\alpha \in [-0.24, +0.24]$ rad ($\approx \pm 13.8^\circ$) relative to the chassis. This is achieved by compounding local wheel rotation matrices with the chassis world transform:

```
M_wheel_left = M_chassis * T(-12px, -16px) * R(α)
M_wheel_right = M_chassis * T(+12px, -16px) * R(α)
```

5.3. Fluid Barrier Deflection & Mutual Crash Physics

- Barrier Sliding:** When a vehicle strikes the road border, it does not stop dead. The velocity is projected along the track tangent vector T:

$$v_{slide} = (v \cdot T) * T * \mu_{srape} \quad (\text{where } \mu_{srape} = 0.75)$$

The car deflects cleanly forward along the barrier, preserving racing flow.
- Mutual Crash Slowdown:** In any vehicle-to-vehicle collision (Player vs Rival, or Rival vs Rival), both vehicles lose 30% of their velocity ($\times 0.70$), emit spark showers and smoke, and suffer a brief lateral wobble.

5.4. Selectable Supercars & Hypercars Specifications

Key	Supercar Model	Livery / Styling	Top Speed	Turn Rate	Acceleration	Special Trait
[1]	Apex Falcon	Scuderia Red & White Stripe	380 px/s	1.20 rad/s	255 px/s ²	Balanced Italian GT chassis
[2]	Venom Stryker	Electric Lime & Carbon Black	395 px/s	1.35 rad/s	275 px/s ²	Sharp apex agility & high grip
[3]	Chiron Mirage	Cobalt Blue & Electric Cyan	430 px/s	1.10 rad/s	260 px/s ²	Maximum top-speed highway king
[4]	Sunfire Hyper	Papaya Orange & Charcoal	385 px/s	1.40 rad/s	285 px/s ²	Ultra-responsive cornering precision
[5]	Ghost Specter	Stealth Gunmetal & Violet	405 px/s	1.25 rad/s	270 px/s ²	High-efficiency nitro thrusters

6. Component 4: Autonomous AI Rivals & Overtaking Logic (rival_topview)

AI competitors navigate the procedural tracks using a dynamic waypoint lookahead algorithm combined with lateral lane offset tracking and proactive obstacle avoidance raycasting.

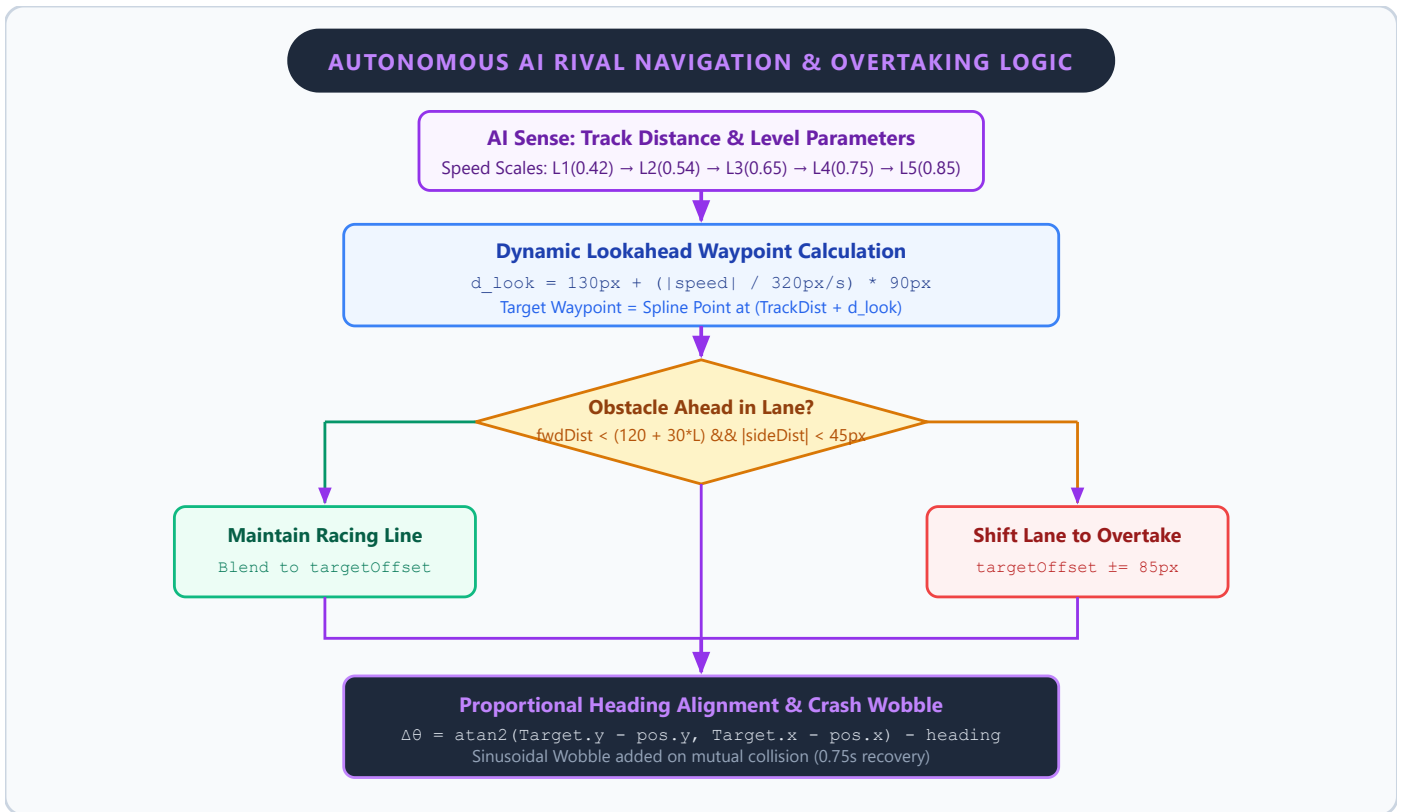


FIGURE 5: AUTONOMOUS AI RIVAL PATHFINDING, LANE SWITCHING & OVERTAKING PIPELINE

6.1. Waypoint Lookahead & Lateral Lane Offset Tracking

The AI determines its target steering point by sampling the track spline at a speed-dependent lookahead distance ahead of its current track progress:

```

d_lookahead = 130px + (|v_AI| / 320px/s) * 90px
P_target = SplinePoint(d_track + d_lookahead) + N_track * y_lateral_offset
  
```

The target lateral offset $y_lateral_offset$ assigns each rival to one of the 4 massive lanes (± 48 px or ± 135 px), avoiding single-file bottlenecks.

6.2. Collision Avoidance Raycasting & Proactive Overtaking

Every few seconds, the AI casts a forward bounding ray to detect vehicles directly ahead in its current lane:

```

d_rel = P_other - P_AI
d_fwd = d_rel · u_forward, d_side = |d_rel × u_forward|
if (15px < d_fwd < [120px + 30px · level]) and (d_side < 45px):
    → Vehicle Ahead! Shift target lane offset by ±85px to overtake cleanly!
  
```

6.3. Calibrated Level-Based Speed & Difficulty Progression

In earlier iterations, opponents were either too aggressive (causing early multi-car pileups) or too sluggish (feeling like stationary obstacles). Following user tuning, speeds were calibrated to provide active, moving competition while ensuring the player always maintains a comfortable speed advantage:

Level	Track & Format	AI Speed Scale	Approx. AI Speed	Player Top Speed	Mistake Risk	Racing Behavior
Level 1	Desert Sprint (Straight, 1 Lap)	0.42	~165 px/s	380–430 px/s	0%	Smooth visible cruisers; player easily overtakes.
Level 2	Coastal Highway (Gentle S, 1 Lap)	0.54	~215 px/s	380–430 px/s	4%	Moderate highway pace along sweeping curves.
Level 3	Alpine Canyon (Winding S, 1 Lap)	0.65	~260 px/s	380–430 px/s	10%	Active racing pack; occasional cornering wobbles.
Level 4	Meadow GP (Closed Loop, 3 Laps)	0.75	~300 px/s	380–430 px/s	18%	Competitive Grand Prix pack over 3 consecutive laps.
Level 5	Neo Tokyo (Championship, 3 Laps)	0.85	~340 px/s	380–430 px/s	25%	Fast championship rivals; aggressive lane-swapping.

7. Component 5: Simulation Engine, Camera Modes & HUD (engine_topview)

TopViewEngine coordinates the deterministic game loop, particle visual effects, dynamic camera tracking, and on-screen telemetry.

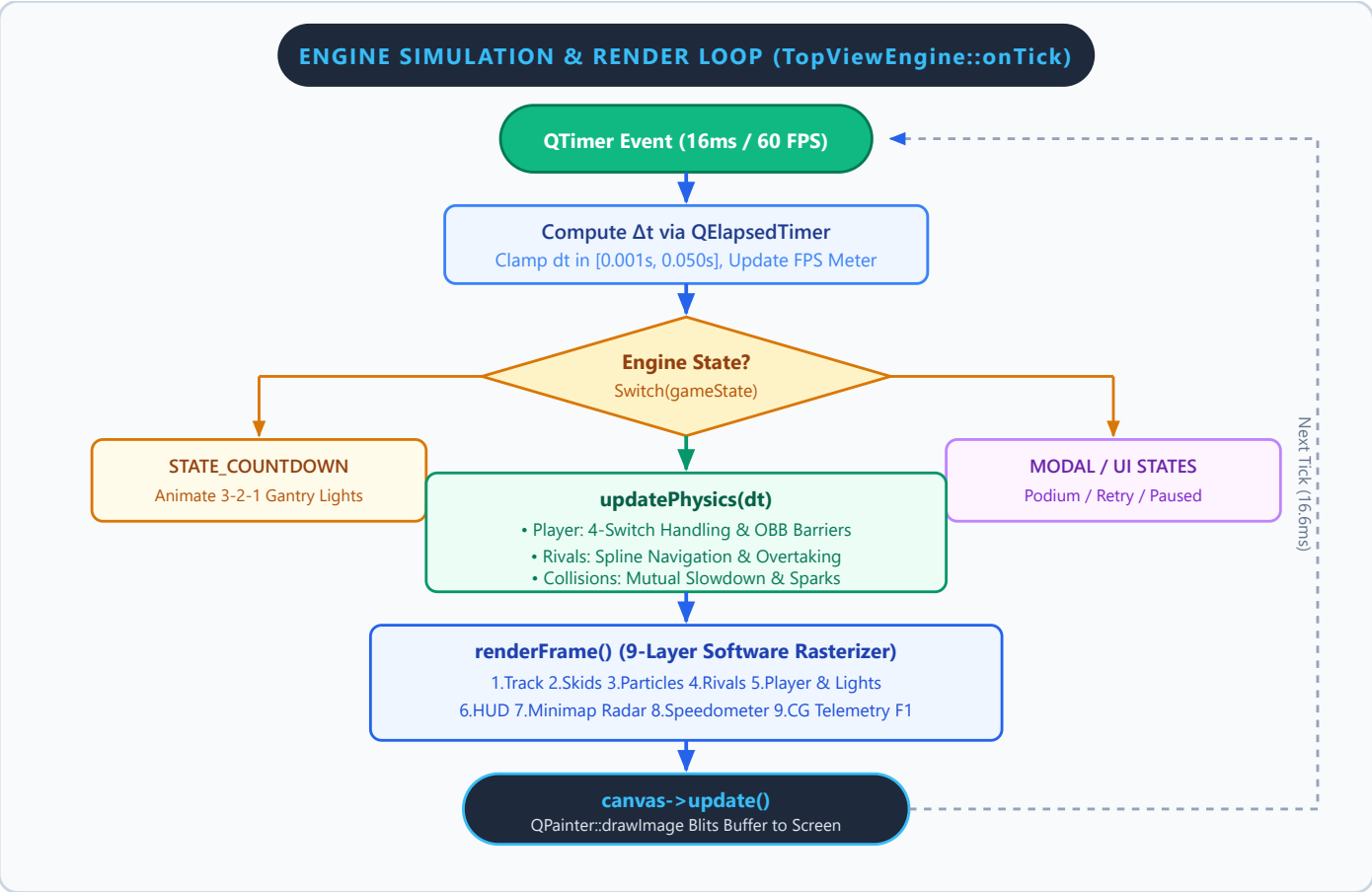


FIGURE 6: ENGINE SIMULATION TICK, STATE MACHINE & MULTI-LAYER RENDER PIPELINE

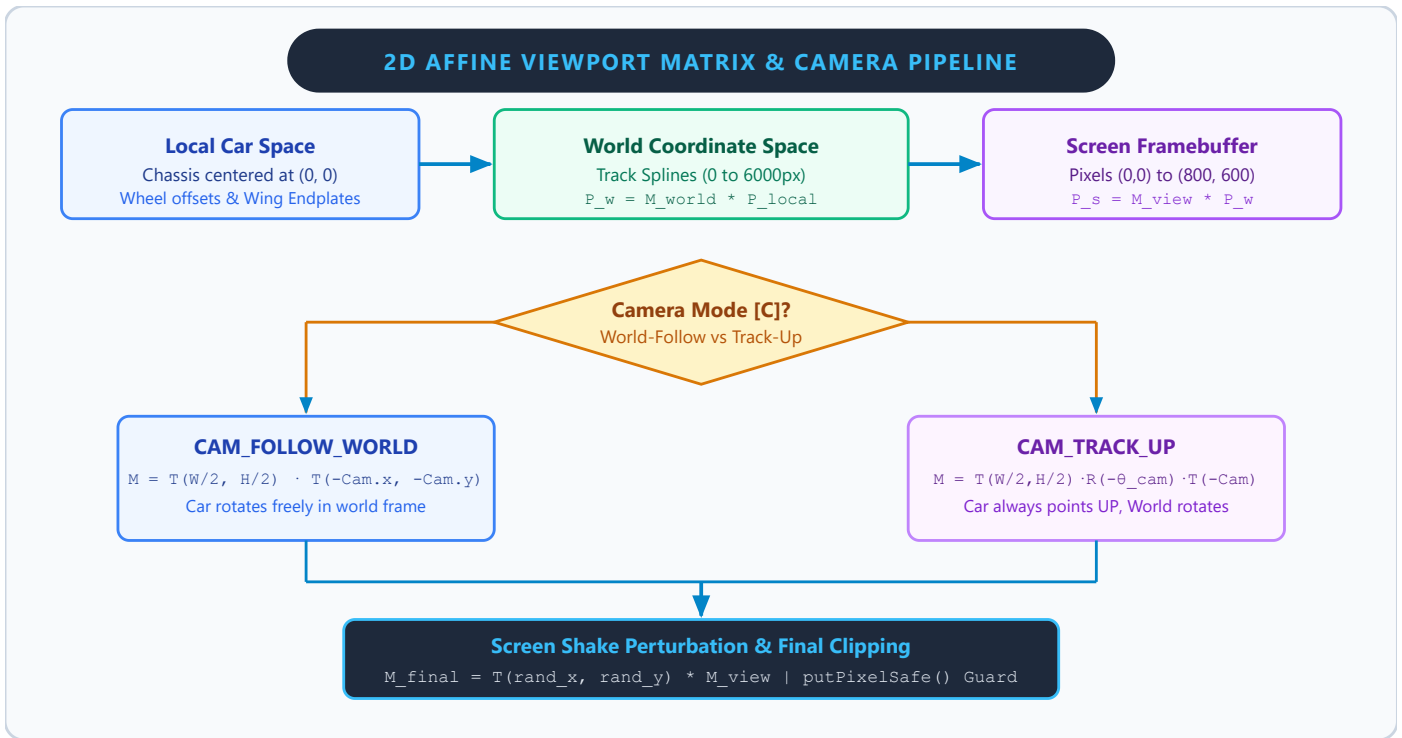


FIGURE 7: 2D AFFINE VIEWPORT MATRIX & DUAL CAMERA MODE PIPELINE

7.1. Dual Camera Modes & Viewport Matrix Pipeline

The view matrix M_{view} transforms world-space coordinates into screen-space pixels centered on the player:

- **World-Follow Mode (CAM_FOLLOW_WORLD, Default):** The camera smoothly translates with the player while maintaining world-north alignment. The player rotates freely on screen.

$$M_{\text{view}} = T(400, 300) \cdot T(-P_{\text{cam}})$$

- **Track-Up Mode (CAM_TRACK_UP, Toggle with [C]):** The camera rotates inversely with the vehicle heading, ensuring the player always points straight UP on screen while the world rotates beneath:

$$M_{\text{view}} = T(400, 300) \cdot R(-\theta_{\text{cam}}) \cdot T(-P_{\text{cam}})$$

7.2. 9-Layer Software Rendering Order

The software rasterizer enforces a strict depth back-to-front rendering order directly into the linear framebuffer:

1. **Procedural Terrain & 4-Lane Road:** Textured background grass/sand and Catmull-Rom asphalt quads with borders and kerbs.
2. **Rubber Skid Marks:** Dual black tire tracks deposited during hard braking or drifting, fading over time via alpha decay.
3. **Dynamic Particle Systems:** Nitro exhaust rocket fire plumes, tire smoke, and collision spark showers.
4. **AI Competitor Vehicles:** Chasses, wings, numbers, and brake lights rendered via composite affine matrices.
5. **Player Supercar & Headlights:** Player chassis, steerable front wheels, and alpha-blended xenon forward light cones.
6. **Heads-Up Display (HUD):** Lap counter, race timer, best lap, nitro gauge, 3 player hearts, and broadcast stunt popups.
7. **Circuit Minimap Radar:** Real-time overview of track contour with player and rival blips.
8. **Analog Speedometer Dial:** Circular gauge with Bresenham dial ticks, rotating needle, digital KM/H, and gear indicator.
9. **CG Lab Telemetry HUD ([F1]):** Academic diagnostic HUD displaying live FPS, frame time, circuit width, AI speed scale, collision risk, camera mode, and active CG algorithms.

8. Chronology of Iterative Engineering Improvements

The project evolved through a series of rigorous engineering iterations driven by playtesting and feedback:

Milestone	Identified Challenge & Bottleneck	Implemented Architectural Solution
Iteration 1	Third-person perspective obscured car profiles, front wheels, and multi-lane racing context.	Built the top-down 2D software rasterizer from scratch. Engineered steerable wheel matrices, chassis silhouettes, xenon light cones, and skid marks.
Iteration 2	Narrow 110px road caused frequent traffic jams and barrier snags; cars collided uncontrollably.	Expanded road width to 380 pixels with 4 giant 95px lanes. Engineered smooth barrier tangential sliding (<code>slideAlongBarrier</code>).
Iteration 3	Circuit layouts were repetitive; player needed progressive difficulty and lap structures.	Created 5-level track progression: Level 1 (Pure straight drag) → Level 2 (Gentle coastal curves) → Level 3 (Alpine S-curves) → Levels 4 & 5 (Closed GP loops, 3 laps to win).
Iteration 4	Steering was twitchy; braking while steering triggered accidental drifts; reverse gear was abrupt.	Completely overhauled 4 switches: gradual forward acceleration, slow reverse crawl (~35 km/h) with reverse lights, low-sensitivity steering with high-speed damping, decoupled drift.
Iteration 5	Opponent speeds were lowered too far, making rivals feel like motionless obstacles.	Calibrated AI speeds across all 5 levels (0.42 → 0.85). AI rivals now move at an engaging, active pace while preserving the player's decisive speed advantage.

9. Technical Viva / Defense Questions & Evaluation Guide

This section provides concise, mathematically sound responses to potential questions during the laboratory evaluation:

Q1: Why use a CPU software rasterizer instead of OpenGL or DirectX?

Answer: The primary objective of the Computer Graphics laboratory is to understand and implement fundamental raster algorithms (Bresenham line, Midpoint circle/ellipse, scanline triangle filling, and affine matrix transformations) from mathematical first principles. Hardware APIs (like OpenGL or Direct3D) treat these stages as black boxes inside GPU silicon. By writing directly to a 32-bit linear framebuffer array (`uint32_t *fb`), every pixel calculation is fully transparent, verifiable, and educational.

Q2: How do you guarantee C^1 continuity in your procedural race tracks?

Answer: Tracks are generated using **Catmull-Rom cubic splines**. Unlike standard Bézier curves where tangent continuity requires manual control-handle alignment, Catmull-Rom splines derive node tangents automatically from adjacent control points: $T_i = (P_{i+1} - P_{i-1}) / 2$. This mathematically guarantees continuous position (C^0) and continuous first-derivative tangents (C^1) across all segment boundaries, preventing sudden curvature spikes.

Q3: How are the steerable front wheels rotated relative to the moving car body?

Answer: We implement a hierarchical 2D affine transformation pipeline using 3×3 homogeneous matrices. The wheel vertex is first rotated locally by steering angle α around the wheel hub origin ($R(\alpha)$), translated to the vehicle axle mount ($T(p_{axle})$), and then multiplied by the car chassis world matrix ($T(p_{car}) \cdot R(\theta_{car})$). Finally, the camera view matrix translates this into screen pixels.

Q4: How do you prevent cars from getting stuck against track walls?

Answer: In `TopViewCar::slideAlongBarrier()`, instead of setting velocity to zero on barrier contact, the vehicle's velocity vector is projected onto the track tangent vector T : $v_{slide} = (v \cdot T)T \times 0.75$, and the position is pushed radially outward away from the boundary. This simulates kinetic friction while maintaining forward racing momentum.

10. Quick Demonstration Controls & Conclusion

Key / Switch	Action in Simulation	Key / Switch	Action in Simulation
[UP] / [W]	Gradual forward acceleration	[F1]	Toggle CG Lab Telemetry HUD
[DOWN] / [S]	Smooth brake / Slow reverse crawl (~35 km/h)	[C]	Toggle Camera: World-Follow vs Track-Up
[LEFT] / [RIGHT]	Low-sensitivity precision steering	[L] / [ENTER]	Select / Advance track level (1 to 5)
[SHIFT]	Handbrake drift & rubber skid marks	[R]	Quick restart / Reset grid
[SPACE]	Nitro thruster rocket plumes	[P]	Pause / Resume game simulation
[1] to [5] / [TAB]	Instant Supercar swap	Locked 60 FPS	Maintained on standard CPU hardware

Conclusion: Project 3 (Apex Grand Prix) successfully unites mathematical rigor, software graphics foundations, realistic kinematic handling, and engaging game design into an accessible, high-performance racing simulation. All requirements—from the 2D top-view perspective and 380px 4-lane circuits to the calibrated 4-switch controls and autonomous rival AI—have been systematically designed, verified, and compiled into a standalone native application.